

Student Meal Planner

Architecture Overview

A Design Document for the Grubs4Scrubs Project

Date: April 2026

Author: Yao Cheng Zhou

LO mapped: LO2 Designing (Software Design)

What this document is

This is a plain-English overview of how the Grubs4Scrubs project is actually structured. Not the theory behind layered architecture (that's in the Layered Architecture Research doc), but the real layout of folders, projects, and files as they exist right now in the solution. If someone opened the repo cold, this is what they'd see.

Solution structure

The solution file sits at the root: `Grubs4Scrubs.sln`. It contains five .NET projects and a React frontend folder. Here's the breakdown.

```
Grubs4Scrubs/
├─ Grubs4Scrubs.sln
├─ Grubs4Scrubs.API/      ← ASP.NET Core Web API (controllers,
Program.cs, config)
├─ Grubs4Scrubs.Business/ ← Service interfaces and implementations
├─ Grubs4Scrubs.DataAccess/ ← ADO.NET repositories
├─ Grubs4Scrubs.Domain/   ← Models, interfaces, DTOs
├─ Grubs4Scrubs.Tests/    ← Unit tests with fake repositories
├─ Grubs4Scrubs.Docs/     ← Portfolio documentation (markdown)
└─ Frontend/             ← React + Vite app
    └─ src/
        ├─ Pages/         ← Page components + CSS
        ├─ Components/    ← Shared components (Navbar, Footer, Searchbar)
        └─ services/      ← Axios API service files
```

Backend layers

Four projects, each with a single job. Dependencies flow downward only.

Grubs4Scrubs.Domain

The bottom layer. No dependencies on anything else in the solution. Contains:

- **Models:** `Recipe.cs`, `User.cs`, `ShoppingItem.cs`. Plain C# classes with properties. No behavior, no database logic, no annotations. Just the shape of the data.
- **Interfaces:** `IRecipeRepository.cs`, `IUserRepository.cs`, `IShoppingItemRepository.cs`. These define what the data access layer must be able to do (GetAll, GetById, Create, Update, Delete) without saying how.

- **DTOs:** `AuthDtos.cs` holds `RegisterDto` and `LoginDto`, which define exactly what the frontend sends during registration and login. They're separate from the User model because the client shouldn't be sending a password hash or a creation timestamp.

Grubs4Scrubs.DataAccess

References Domain only. Contains the concrete repository implementations:

- `RecipeRepository.cs`, `UserRepository.cs`, `ShoppingItemRepository.cs`. Each one opens a `SqlConnection`, runs parameterized `SqlCommand` queries, and maps `SqlDataReader` rows to domain model objects. Every query uses `@Parameter` placeholders to prevent SQL injection.
- `DbSeeder.cs` for populating the database with sample data during development.

No Entity Framework. No LINQ-to-SQL. Everything is raw ADO.NET, which is a project requirement. The trade-off is more boilerplate code, but it also means I understand exactly what SQL is running and when.

Grubs4Scrubs.Business

References Domain only (same as DataAccess). Contains service interfaces and implementations:

- `IRecipeService.cs` / `RecipeService.cs`
- `IUserService.cs` / `UserService.cs`
- `IShoppingItemService.cs` / `ShoppingItemService.cs`

Each service takes a repository interface through its constructor (dependency injection). The service layer handles validation (is the title empty? is the budget negative?) and any business rules before calling the repository. Controllers never touch the database directly.

Grubs4Scrubs.API

The top layer. References Business and DataAccess (to register DI services) and Domain (to use the models). Contains:

- **Controllers:** `RecipeController.cs`, `UserController.cs`, `ShoppingItemController.cs`, `AuthController.cs`. Each controller is thin. It receives an HTTP request, calls the appropriate service method, and returns a response. No SQL, no validation logic, no business rules.

- **Program.cs:** The startup file. Configures dependency injection (registers all services and repositories), sets up CORS for the Vite dev server, configures JWT authentication middleware, and defines the middleware pipeline order.
- **appsettings.json:** Holds the database connection string and JWT configuration (key, issuer, audience).

Grubs4Scrubs.Tests

References Domain and Business. Uses a `FakeRepository` that implements the repository interfaces with in-memory lists. This lets me test service-layer logic without needing a real database running.

How a request flows

A concrete example: fetching all recipes.

1. React frontend calls `axios.get("http://localhost:5075/api/recipes")`
2. Request hits `RecipeController.GetAll()`
3. Controller calls `_recipeService.GetAllRecipes()`
4. Service calls `_recipeRepository.GetAll()`
5. Repository opens a `SqlConnection`, runs `SELECT * FROM Recipes`, maps each row to a `Recipe` object
6. Objects flow back up: Repository returns `List<Recipe>` to Service, Service returns it to Controller, Controller returns 200 OK with JSON

The auth flow is slightly different. `AuthController` doesn't go through a service layer because it handles its own logic directly: BCrypt password hashing for registration, BCrypt verification plus JWT token generation for login. This might get refactored into an `IAuthService` later, but right now it's self-contained in the controller.

Frontend architecture

The React app follows a simple flat structure. No nested routing, no global state management, no component library.

Pages

Each page is a pair: `PageName.jsx` + `PageName.css`. The pages are:

- `HomePage` (landing page with hero, features, featured recipes, CTA)
- `DashboardPage` (stats cards, today's meals, mini shopping list)

- `RecipesPage` (filterable recipe grid with category buttons, add recipe modal)
- `RecipeViewPage` (full recipe detail with ingredients checklist, edit modal, delete)
- `MealPlannerPage` (weekly planner with day tabs and meal slots)
- `ShoppingListPage` (shopping items with inline editing, check-off, delete)
- `LoginPage` (login form UI, not yet connected to API)
- `SignUpPage` (registration form UI, not yet connected to API)

Shared components

- `Navbar.jsx` (top navigation bar, present on every page)
- `Footer.jsx` (footer bar, present on every page)
- `Searchbar.jsx` (search input component)

API service layer

`services/recipeService.js` wraps Axios calls to the backend. Base URL is `http://localhost:5075/api`. Currently covers recipe and shopping item CRUD. User/auth service files haven't been created yet since the frontend login isn't wired up.

CSS approach

Custom CSS with a strict naming convention: `ComponentName-element-subelement`. No Tailwind (it was stripped out earlier in the semester after the utility classes made the JSX unreadable). Every page has its own `.css` file. Shared variables like colors and border radii live in `index.css`.

Authentication architecture

The auth system is backend-only for now. Here's what exists:

- `AuthController` with `/api/auth/register` and `/api/auth/login` endpoints
- BCrypt password hashing on registration via `BCrypt.Net.BCrypt.HashPassword()`
- BCrypt verification on login via `BCrypt.Net.BCrypt.Verify()`
- JWT token generation with claims (user ID, email, username) signed with HMAC-SHA256
- JWT middleware in `Program.cs` that validates tokens on incoming requests
- `RegisterDto` and `LoginDto` in the Domain layer so the API accepts clean input shapes

The frontend has LoginPage and SignUpPage components built, but they aren't connected to these endpoints yet. That's the next submission's scope.

Database

SQL Server, accessed through raw ADO.NET. Three tables:

- **Recipes** (Id, Title, Description, PrepTime, CookTime, Servings, Tag, ImageUrl, EstimatedBudget, Category, CreatedAt, plus JSON-stored Ingredients, Instructions, Tips, and nutrition fields)
- **Users** (Id, Email, PasswordHash, UserName, GoogleId, CreatedAt)
- **ShoppingItems** (Id, Name, Quantity, Price, IsChecked, UserId)

The ERD and table details are in a separate document.

What's not built yet

Being honest about what's missing:

- Frontend auth wiring (LoginPage/SignUpPage to API)
- Meal plan persistence (currently client-side only)
- Shopping list tied to user accounts
- Recipe ownership (only the creator should edit/delete their recipes)
- Error handling middleware (currently basic try/catch in controllers)
- Production deployment setup

[NOTE: If there are other features you've started or planned that I missed, add them here.]